

NestJS kézikönyv (magyar)

Ez a dokumentum átfogó, magyar nyelvű útmutató a NestJS fő témaköreiről: Overview, Fundamentals, Techniques, Security, GraphQL, Websockets és Microservices. Tartalmaz javasolt példákat és kódrészleteket. A fájl UTF-8 kódolású.

Tartalomjegyzék

1. Overview — Áttekintés
2. Fundamentals — Alapok
3. Techniques — Technikák
4. Security — Biztonság
5. GraphQL
6. Websockets
7. Microservices — Mikroszolgáltatások
8. Kiegészítők: Docker, Swagger, Tesztek

1) Overview — Áttekintés

Mi a NestJS?

- A NestJS egy progresszív, Node.js alapú szerveroldali keretrendszer, amely TypeScript-re épülve segíti a skálázható, karbantartható alkalmazások fejlesztését.
- Célja, hogy vállalati szintű felépítést és szervezést adjon a Node-alkalmazásokhoz, miközben használhatod az ismert JavaScript/TypeScript ökoszisztéma eszközeit (Express vagy Fastify, ORM-ek, WebSocket-könyvtárak, stb.).
- Inspirációt merít az Angular szerkezetéből (modulok, injektálható szolgáltatások, dekorátor-alapú deklaráció).

Főbb gondolatok

- Modularitás: a kódot modulokra bontva könnyebb tesztelni és újrafelhasználni.
- Dependency injection (DI): beépített IoC konténer a laza csatolásért és egyszerű tesztelhetőségért.
- Dekorátorok használata: TypeScript-dekorátorokkal (pl. @Controller, @Module, @Injectable) deklaratív felépítés.
- Platform-agnosztikus: alapértelmezett HTTP szerver Express, de Fastify is támogatott; lehetőség más platformokra való futtatásra is.

Gyors kezdés (CLI)

```
npm i -g @nestjs/cli
nest new project-name
```

```
cd project-name
npm run start:dev
```

2) Fundamentals — Alapok

Alapfogalmak és architektúra

- Module (@Module): a Nest alkalmazás szervező egysége. Minden modul deklarálja a controllereket, providereket (szolgáltatásokat) és exportált elemeket.
- Controller (@Controller): kezeli a bejövő kéréseket és válaszokat — tipikusan HTTP útvonalakhoz kötődik.
- Provider / Service (@Injectable): üzleti logika, adatkezelés; injektálható komponensek.
- Dependency Injection: konstruktor-injektálással történik, a konténer gondoskodik az életciklus-kezelésről.

Egyszerű alkalmazás példa

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { CatsModule } from '../cats/cats.module';

@Module({
  imports: [CatsModule],
})
export class AppModule {}

// src/cats/cats.module.ts
import { Module } from '@nestjs/common';
import { CatsService } from '../cats.service';
import { CatsController } from '../cats.controller';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}

// src/cats/cats.service.ts
import { Injectable } from '@nestjs/common';

export type Cat = { id: number; name: string; age?: number };

@Injectable()
export class CatsService {
```

```

    private readonly cats: Cat[] = [{ id: 1, name: 'Mici' }];

    findAll(): Cat[] {
        return this.cats;
    }

    create(cat: Cat) {
        this.cats.push(cat);
        return cat;
    }
}

// src/cats/cats.controller.ts
import { Controller, Get, Post, Body } from '@nestjs/common';
import { CatsService, Cat } from './cats.service';

@Controller('cats')
export class CatsController {
    constructor(private readonly catsService: CatsService) {}

    @Get()
    getAll() {
        return this.catsService.findAll();
    }

    @Post()
    create(@Body() cat: Cat) {
        return this.catsService.create(cat);
    }
}

```

Middleware

```

// src/logger.middleware.ts
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
    use(req: Request, res: Response, next: NextFunction) {
        console.log(`${req.method} ${req.originalUrl}`);
        next();
    }
}

// app.module.ts használat:

```

```
// consumer.apply(LoggerMiddleware).forRoutes('*');
```

Pipes (ValidationPipe példa)

```
// src/users/dto/create-user.dto.ts
import { IsString, IsEmail, IsOptional, IsInt } from 'class-validator';

export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;

  @IsOptional()
  @IsInt()
  age?: number;
}

// main.ts
import { ValidationPipe } from '@nestjs/common';
app.useGlobalPipes(new ValidationPipe({ transform: true, whitelist: true }));
```

Guards (Role példa)

```
// src/auth/roles.decorator.ts
import { SetMetadata } from '@nestjs/common';
export const Roles = (...roles: string[]) => SetMetadata('roles', roles);

// src/auth/roles.guard.ts
import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}
  canActivate(context: ExecutionContext): boolean {
    const required = this.reflector.get<string[]>('roles', context.getHandler()) || [];
    if (!required.length) return true;
    const req = context.switchToHttp().getRequest();
    const user = req.user;
    return user && required.some(r => user.roles?.includes(r));
  }
}

// használat controlleren
```

```
// @UseGuards(RolesGuard)
// @Roles('admin')
// @Get('admin')
// adminOnly() { return 'secret'; }
```

Interceptor (logging) példa

```
// src/common/interceptors/logging.interceptor.ts
import {
  Injectable,
  NestInterceptor,
  ExecutionContext,
  CallHandler,
} from '@nestjs/common';
import { Observable, tap } from 'rxjs';

@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const now = Date.now();
    return next.handle().pipe(
      tap(() => console.log(`${context.switchToHttp().getRequest().url} - ${Date.now() - now}`)),
    );
  }
}
```

Exception filter példa

```
// src/common/filters/http-exception.filter.ts
import { ExceptionFilter, Catch, ArgumentsHost, HttpException } from '@nestjs/common';
import { Request, Response } from 'express';

@Catch(HttpException)
export class HttpErrorFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const res = ctx.getResponse<Response>();
    const req = ctx.getRequest<Request>();
    const status = exception.getStatus();
    const error = exception.getResponse();
    res.status(status).json({
      statusCode: status,
      timestamp: new Date().toISOString(),
      path: req.url,
      error,
    });
  }
}
```

}
}